

A Uniform Lower Bound for Syracuse Atoms

Mathematical derivation and checked Lean source

Reverification: 12 September 2026

Abstract

We derive a constant $c > 0$, independent of the depth n and the unit residue $r \pmod{3^n}$, such that the actual Syracuse probability satisfies $\mu_n(r) \geq c3^{-n}$. The connection is a positive-part comparison of filtered backward marks with the full law, followed by a fixed-cylinder argument and a bounded inverse step. We give the complete derivation of this connection, state precisely the imported analytic inputs, explain their summability mechanism, and append the consolidated Lean proof that was compiled for this document. The argument uses substantial prior work by Terence Tao and Lech Mazur. It does not prove the Collatz conjecture or establish historical priority.

Contents

1	The exact theorem and the probability law	2
2	Normalization, projectivity, and the source inputs	2
3	Prefix transport and the one-sided estimate	3
4	Filtered kernels and iteration	4
5	The core inputs and why their tails are uniform	5
6	Freezing one positive cylinder	6
7	Vanishing error and projection onto a fixed subcell	6
8	Bounded inverse coverage of the fixed cylinder	7
9	Proof of the uniform bound at every depth	8
10	Verification, dependency boundaries, and reproduction	8
A	Complete consolidated Lean proof	10

1 The exact theorem and the probability law

Let $R_n = \mathbb{Z}/3^n\mathbb{Z}$ and $U_n = R_n^\times$. If $h \leq q$, write $\pi_{h,q} : R_q \rightarrow R_h$ for reduction modulo 3^h . Let $A_1, A_2, \dots$ be independent positive integer variables satisfying

$$\mathbb{P}(A_i = v) = 2^{-v}, \quad v = 1, 2, \dots \quad (1)$$

All powers of 2 below are inverted in the indicated odd-modulus ring. Define the probability mass function

$$\mu_n(r) = \mathbb{P} \left(\sum_{j=1}^n 3^{j-1} 2^{-(A_1 + \dots + A_j)} = r \text{ in } R_n \right). \quad (2)$$

The empty sum at level zero is zero. The reversed iid list has the same law, so this is equivalently the recursive law used in Lean:

$$F_{n,v}(z) = 2^{-v}(3\tilde{z} + 1) \pmod{3^{n+1}}, \quad \mu_{n+1}(r) = \sum_{z \in R_n} \sum_{v \geq 1} \mu_n(z) 2^{-v} \mathbf{1}_{F_{n,v}(z)=r}. \quad (3)$$

Here $\tilde{z}$ is the canonical integer representative. Changing its lift by 3^n changes $3\tilde{z}$ by 3^{n+1} , so the map is well-defined. The Lean objects are `Erdos1135.Tao.syracStep` and `Erdos1135.Tao.syracPMF`.

Theorem 1.1 (Uniform atom lower bound). *There exists a real number $c > 0$ such that for every integer $n \geq 1$ and every $r \in U_n$,*

$$\mu_n(r) \geq \frac{c}{3^n}. \quad (4)$$

The quantifier order is $\exists c \ \forall n \ \forall r$. No hypothesis about deterministic Collatz convergence is present.

For $n \geq 1$, the law is supported on U_n : reducing the sum in (2) modulo 3 leaves $2^{-A_1} \neq 0$. There are $2 \cdot 3^{n-1}$ units. Consequently, if $m_n = \min_{r \in U_n} \mu_n(r)$, averaging and Theorem 1.1 give

$$c3^{-n} \leq m_n \leq \frac{3}{2} 3^{-n}. \quad (5)$$

In particular $m_n = \Theta(3^{-n})$, and $\log m_n = -n \log 3 + O(1)$, hence $m_n = 3^{-n+o(n)}$. This is stronger than the exponential-rate conjecture discussed in Tao's 2020 note [2]. The asymptotic interpretation is an elementary corollary explained here; the literal audited Lean endpoint is (4).

2 Normalization, projectivity, and the source inputs

For a function $f : R_q \rightarrow \mathbb{R}$, put

$$\langle f \rangle_q = 3^{-q} \sum_{y \in R_q} f(y), \quad \|f\|_{1,q} = \langle |f| \rangle_q, \quad t_+ = \max(t, 0).$$

The density used in the imported core library is

$$\rho_q(y) = \frac{2}{3} 3^q \mu_q(y), \quad \langle \rho_q \rangle_q = \frac{2}{3}. \quad (6)$$

This is density relative to unit Haar measure, extended by zero, for positive q . We still take all displayed averages over the *full* ring R_q . At $q = 0$, the same definition gives $\rho_0(0) = 2/3$.

Reduction in (2) discards all terms indexed above h . Therefore

$$\sum_{\pi_{h,q}(y)=x} \mu_q(y) = \mu_h(x), \quad \sum_{\pi_{h,q}(y)=x} \rho_q(y) = 3^{q-h} \rho_h(x). \quad (7)$$

Each fiber has 3^{q-h} points. In particular $\langle f \circ \pi_{h,q} \rangle_q = \langle f \rangle_h$.

We use the following imported fine-scale mixing theorem [1, 3]: for every positive integer A , there exists $C_A \geq 0$ such that

$$\|\rho_q - \rho_k \circ \pi_{k,q}\|_{1,q} \leq \frac{2C_A}{3k^A}, \quad 1 \leq k \leq q. \quad (8)$$

It is `Tao.taoProp114FineScaleMixing` together with the exact normalization lemma for the reference density.

The second major input is Mazur's core construction and its uniform variation estimate. Section 5 specifies precisely the facts used and the quantitative majorant yielding their convergence. These are *proved library theorems*, not added assumptions in the final Lean statement. This document does not reproduce the proofs of the entire imported Fourier and core library; it gives every step from these explicitly identified results to Theorem 1.1. The supporting source is pinned and included in the local verification scan.

3 Prefix transport and the one-sided estimate

For a finite valuation word $w = (a_1, \dots, a_d)$, write $|w| = d$, $s(w) = \sum_i a_i$, and $p(w) = 2^{-s(w)}$. Use the outer-first composition convention

$$\Phi_w(z) = \phi_{a_1} \circ \dots \circ \phi_{a_d}(z), \quad \phi_v(z) = 2^{-v}(3z + 1).$$

Thus the constant term of Φ_w is the finite sum in (2). On $R_{q-d} \rightarrow R_q$, Φ_w is injective: its linear coefficient is $3^d 2^{-s(w)}$. Define the weighted density transport

$$(T_{w,q}f)(y) = 3^d p(w) \sum_{\substack{z \in R_{q-d} \\ \Phi_w(z)=y}} f(z). \quad (9)$$

Injectivity, followed by summation, gives

$$\|T_{w,q}f\|_{1,q} = p(w) \|f\|_{1,q-d}. \quad (10)$$

This positive linear map is the mathematical form of the imported reference prefix transport.

Let $\mathcal{W}$ be a finite prefix-free family with $|w| \leq q$. Its prefix events are pairwise disjoint under the product law (1), so

$$\sum_{w \in \mathcal{W}} p(w) \leq 1. \quad (11)$$

Attaching the full residual Syracuse law after each word yields a selected subdensity of the full law:

$$S(y) := \sum_{w \in \mathcal{W}} T_{w,q} \rho_{q-|w|}(y) \leq \rho_q(y). \quad (12)$$

The factor $2/3$ in (6) is the same for all residual lengths, including zero, so the normalization is consistent.

Suppose $1 \leq k \leq q - |w|$ for all words, and replace each residual density by its k -digit projection. Set

$$L(y) = \sum_{w \in \mathcal{W}} T_{w,q}(\rho_k \circ \pi_{k,q-|w|})(y).$$

By (10), (8), and (11),

$$\begin{aligned} \|L - S\|_{1,q} &\leq \sum_{w \in \mathcal{W}} p(w) \|\rho_k \circ \pi_{k,q-|w|} - \rho_{q-|w|}\|_{1,q-|w|} \\ &\leq \frac{2C_A}{3k^A}. \end{aligned} \quad (13)$$

Since $S \leq \rho_q$, pointwise $(L - \rho_q)_+ \leq |L - S|$, whence

$$\langle (L - \rho_q)_+ \rangle_q \leq \frac{2C_A}{3k^A}. \quad (14)$$

The useful change of estimate: rejected words are favorable to a lower bound. Their total mass is not an error term in (14). This is why the positive part is used instead of absolute error against the full law.

4 Filtered kernels and iteration

For an imported selected prefix family of maximum horizon H , a terminal conductor k , and a word filter $0 \leq \psi(w) \leq 1$, define

$$Kf = \sum_{w \in \mathcal{W}} \psi(w) T_{w,H+k}(f \circ \pi_{k,H+k-|w|}). \quad (15)$$

Positivity gives $(Kf - Kg)_+ \leq K((f - g)_+)$. The mean identity from (10) gives, for $u \geq 0$,

$$\langle Ku \rangle_{H+k} = \left(\sum_w \psi(w) p(w) \right) \langle u \rangle_k \leq \langle u \rangle_k.$$

Consequently

$$\langle (Kf - Kg)_+ \rangle_{H+k} \leq \langle (f - g)_+ \rangle_k. \quad (16)$$

Also $K\rho_k \leq L$ because $\psi \leq 1$ and the transported densities are nonnegative. Absorbing $2/3$ into the constant in (14),

$$\langle (K\rho_k - \rho_{H+k})_+ \rangle_{H+k} \leq D_A/k^A. \quad (17)$$

The constant is uniform over the root-side parameters and the selected families. It is not chosen separately at each generation.

For a base $b \geq 200$ define $F(b) = b + \lfloor b/100 \rfloor$. The imported horizon is $H(b) = b + \lfloor 3b/5 \rfloor$. With arbitrary cap and width schedules, define recursively

$$\begin{aligned} q(b, k, 0) &= k, & G(b, k, 0) &= \rho_k, \\ q(b, k, N+1) &= H(b) + q(F(b), k, N), & G(b, k, N+1) &= K_b G(F(b), k, N). \end{aligned}$$

In the last line the cap schedule is shifted by one, as in the Lean recursion. The core word filter is the indicator

$$\psi_{b,m}(w) = \mathbf{1}_{\{b \leq |w|+m, |w| \leq b+m\}}.$$

The selected family itself is the imported capped least-crossing word family; it is not selected using the terminal density. Its exact source name is `ndShiftedReferenceSelectedWords`. The results above need its prefix-free property, bounded horizon, and original probabilities only.

Lemma 4.1 (All-generation comparison). *For every positive integer A there exists $D_A \geq 0$ such that*

$$\langle (G(b, k, N) - \rho_{q(b, k, N)})_+ \rangle_{q(b, k, N)} \leq ND_A/k^A \quad (k \geq 1).$$

Proof. The case $N = 0$ has error zero. For the inductive step put $t = q(F(b), k, N) \geq k$ and split

$$(K_b G - \rho_{H(b)+t})_+ \leq (K_b G - K_b \rho_t)_+ + (K_b \rho_t - \rho_{H(b)+t})_+.$$

Average and use (16), the induction hypothesis, and (17). The result is at most $ND_A/k^A + D_A/t^A \leq (N+1)D_A/k^A$. $\square$

5 The core inputs and why their tails are uniform

Fix once and for all

$$b_0 = 32^5, \quad b_{j+1} = F(b_j), \quad \text{cap}(j) = \lfloor j/100 \rfloor, \quad m(b) = \lceil b^{3/5} \rceil.$$

For each N , let

$$k_N = \lfloor b_N/4 \rfloor, \quad q_N = q(b_0, k_N, N), \quad G_N = G(b_0, k_N, N)$$

with these cap and width schedules. The use of k_N at every level of one backward recursion is important: it is a fixed terminal conductor for that recursion, although it changes when N changes.

Write s_j for the probability that the imported least-crossing word at floor b_j , with cap $\lfloor j/100 \rfloor$, lies in the narrow strip $[b_j - m(b_j), b_j + m(b_j)]$, and set $P_N = \prod_{j < N} s_j$. The imported core results provide the following facts.

- C1.** $0 < s_j \leq 1$, $\sum_j (1 - s_j) < \infty$, and hence $P_N \geq p > 0$ for every N .
- C2.** $G_N \geq 0$, it vanishes on nonunits, and $\langle G_N \rangle_{q_N} = (2/3)P_N$. In particular some unit y_N satisfies $G_N(y_N) \geq (2/3)P_N$.
- C3.** For every odd integer $M \geq 16^{b_0}$ there exists a real ℓ_M such that

$$|\ell_M - G_N(M \bmod 3^{q_N})| \leq T_N, \quad T_N \longrightarrow 0, \quad (18)$$

where the *same* sequence T_N works for all such M .

Here is the summability mechanism behind C1 and C3, so that the root uniformity is explicit. Let

$$C_{\text{strip}} = 8 \cdot 35! \cdot 576^{35}, \quad g = 2013/2000, \quad r_1 = g(200/201)^6, \quad r_2 = g2^{-1/100}.$$

Both $0 < r_1, r_2 < 1$ are checked in the supporting library. The imported strip and cap bounds imply

$$0 \leq 1 - s_j \leq \frac{C_{\text{strip}}}{b_0^6} (200/201)^{6j} + 2^{-j/100}. \quad (19)$$

Thus the deficits are summable. Since every $s_j > 0$, eventually $s_j \geq 1/2$; then $-\log s_j \leq 2(1 - s_j)$. Therefore $p = \exp(\sum_j \log s_j) > 0$ and $P_N \geq p$. No numerical lower estimate for p is claimed.

For some fixed $C \geq 0$, define

$$B_j = \frac{2}{3} (4b_0^{3/5} + 1) 6(2^{b_0+1} + 16^{b_0})(j+1)^3 \cdot \left[\frac{2C8^6 + C_{\text{strip}}}{b_0^6} r_1^j + r_2^j \right]. \quad (20)$$

Mazur's increment theorem bounds the absolute difference between consecutive core observables by the original state denominator times B_j . For a singleton root that denominator is exactly 1, *independently of M* . The singleton observable is exactly $G_j(M \bmod 3^{q_j})$, not an approximation to it. Since $\sum_j (j+1)^3 r_i^j < \infty$, we may take

$$T_N = \sum_{j \geq N} B_j.$$

The Cauchy criterion and telescoping prove (18). The constant C , all factors in (20), and the tail cutoff are chosen before the physical root. This is the extra information that an arbitrary L^1 -mixing family need not possess.

For direct source correspondence, C1 uses `exists_pos_le_coreProbabilityProduct`; C2 uses `exists_unit_rootCoreBackwardMark_ge_product`; C3 uses `exists_coreMarkedLimit_with_root_uniform_tail`, `rootCoreSingletonState_denominator_eq_one`, and `rootCoreSingletonState_markedMass_eq_backwardMark`. These are in the imported core backward-mean and summed-variation modules.

6 Freezing one positive cylinder

Lemma 6.1. *There exist $q_* \geq 1$, $y_* \in U_{q_*}$, $a > 0$, and N_* such that for every $N \geq N_*$ with $q_N \geq q_*$ and every $z \in R_{q_N}$,*

$$\pi_{q_*, q_N}(z) = y_* \implies G_N(z) \geq a.$$

Proof. Choose N_* so that $T_j < p/6$ for all $j \geq N_*$. By C2 choose $y_* \in U_{q_{N_*}}$ with $G_{N_*}(y_*) \geq 2p/3$ and put $q_* = q_{N_*}$, $a = p/3$. For every eligible $M \equiv y_* \pmod{3^{q_*}}$,

$$\begin{aligned} G_N(M) &\geq \ell_M - T_N \\ &\geq G_{N_*}(M) - T_{N_*} - T_N > 2p/3 - p/6 - p/6 = p/3. \end{aligned}$$

All function arguments here are reduced at their respective conductors. For any fixed finite $z \in R_{q_N}$ above y_* , choose an odd natural representative M of z with $M \geq 16^{b_0}$. Such representatives exist arbitrarily far out because 3^{q_N} is odd: among consecutive integer lifts, parity alternates. Applying the preceding inequality to this M gives the claimed finite-quotient assertion. $\square$

The representative may depend on N and z . This causes no dependence in a or T_N , since (18) is uniform over every eligible representative. The favorable residue itself is chosen only once.

7 Vanishing error and projection onto a fixed subcell

For $b \geq 200$, $\lfloor b/100 \rfloor \geq 2$. Thus $b_N \geq b_0 + 2N$. Using the remainder bound $b_N \leq 4k_N + 3$ gives

$$b_0 + 2N \leq 4k_N + 3, \quad N \leq 2k_N, \quad q_N \geq k_N \geq 1.$$

Lemma 4.1, with $A = 2$, yields for $N > 0$

$$\varepsilon_N := \langle (G_N - \rho_{q_N})_+ \rangle_{q_N} \leq \frac{ND_2}{k_N^2} \leq \frac{4D_2}{N} \longrightarrow 0. \quad (21)$$

Only linear growth is needed here; stronger geometric floor estimates were used in the imported summability analysis.

Lemma 7.1 (Fixed-fiber estimate). *Let $h \leq q$, $x \in R_h$, and suppose $f(z) \geq a$ whenever $\pi_{h,q}(z) = x$. Then*

$$a \leq \rho_h(x) + 3^h \langle (f - \rho_q)_+ \rangle_q.$$

Proof. For every point of the fiber, $a \leq \rho_q(z) + (f(z) - \rho_q(z))_+$. Sum over that fiber and bound its nonnegative error by the error over the full ring. By (7),

$$3^{q-h}a \leq 3^{q-h}\rho_h(x) + 3^q \langle (f - \rho_q)_+ \rangle_q.$$

Division by the positive fiber cardinality proves the assertion. $\square$

Proposition 7.2 (A positive cylinder for the actual law). *For q_* , y_* , a from Lemma 6.1, every $h \geq q_*$ and every $x \in R_h$ with $\pi_{q_*,h}(x) = y_*$ satisfy $\rho_h(x) \geq a$.*

Proof. Fix h and x first. For all sufficiently large N , $N \geq N_*$ and $q_N \geq h$, and $G_N \geq a$ on the entire fiber above x . Thus Lemma 7.1 and (21) give

$$a \leq \rho_h(x) + 3^h \varepsilon_N \leq \rho_h(x) + \frac{4D_2 3^h}{N}.$$

Letting $N \rightarrow \infty$ proves the result. Equivalently, if $\delta = a - \rho_h(x) > 0$, choose an integer $N > \max(N_* + 2h + 1, 4D_2 3^h / \delta)$; the same inequality contradicts $\delta > 0$. This finite contradiction is the implementation in Lean. $\square$

The factor is 3^h , not a harmless uniform constant in h . The proof works because the test conductor h is fixed before N is enlarged. No assertion is inferred by multiplying a moving-conductor L^1 error by its exponentially growing cardinality.

8 Bounded inverse coverage of the fixed cylinder

For $t, c \in \mathbb{N}$ put

$$d_t = (4^t - 1)/3, \quad c_t = 4^t c + d_t = (3c + 1)d_t + c.$$

These are natural numbers and

$$3c_t + 1 = 4^t(3c + 1). \tag{22}$$

The residues d_t , $0 \leq t < 3^q$, permute R_q . Indeed, for $s < t$,

$$d_t - d_s = 4^s \frac{4^{t-s} - 1}{3}, \quad v_3(d_t - d_s) = v_3(t - s) < q.$$

The identity $v_3(4^m - 1) = 1 + v_3(m)$ follows by induction on $v_3(m)$: for $3 \nmid u$, the geometric sum $(4^u - 1)/3$ is $u \not\equiv 0 \pmod{3}$; and if $x \equiv 1 \pmod{3}$, then $x^2 + x + 1$ has valuation exactly one. Factor $x^3 - 1 = (x - 1)(x^2 + x + 1)$ repeatedly. This proves distinctness, and cardinality gives surjectivity. Since $3c + 1$ is invertible modulo 3^q , the affine formula for c_t also permutes all residues. The corresponding existing Lean result is `CollatzTargetDensity.sibling_residue`.

Lemma 8.1 (Bounded inverse valuation). *If $q \leq h$, $y \in R_q$, and $r \in U_{h+1}$, there are $1 \leq v \leq 2 \cdot 3^q + 2$ and $z \in R_h$ with $\pi_{q,h}(z) = y$ and $F_{h,v}(z) = r$.*

Proof. Let R be the canonical representative of r . If $R \equiv 1 \pmod{3}$, take $e = 2$ and $c = (4R - 1)/3$; if $R \equiv 2 \pmod{3}$, take $e = 1$ and $c = (2R - 1)/3$. In both cases $c \in \mathbb{N}$ and $3c + 1 = 2^e R$. Choose $0 \leq t < 3^q$ so that $c_t \equiv y \pmod{3^q}$. Then (22) gives $3c_t + 1 = 2^{2t+e} R$. Set $v = 2t + e$ and $z = c_t \pmod{3^h}$. Reducing c_t before multiplying by 3 preserves the equation modulo 3^{h+1} . Division by 2^v proves $F_{h,v}(z) = r$, and $v \leq 2 \cdot 3^q + 2$. $\square$

No assertion about whether r converges is used. This is finite residue arithmetic, and the bound depends on the prescribed cylinder only.

9 Proof of the uniform bound at every depth

Apply Lemma 8.1 to the cylinder from Proposition 7.2. Set

$$V = 2 \cdot 3^{q_*} + 2, \quad d = 3 \cdot 2^{-V} a > 0.$$

If $h \geq q_*$ and $r \in U_{h+1}$, select the inverse child z above y_* . A single nonnegative term of (3) yields

$$\mu_{h+1}(r) \geq 2^{-v} \mu_h(z).$$

Multiplying by $(2/3)3^{h+1}$, Proposition 7.2 gives

$$\rho_{h+1}(r) \geq 3 \cdot 2^{-v} \rho_h(z) \geq 3 \cdot 2^{-V} a = d. \quad (23)$$

This is a lower bound for all units at every depth at least $q_* + 1$. For $1 \leq h < q_* + 1$, fix a unit $r \in R_h$. Every lift of r in R_{q_*+1} is a unit. By (7), $\rho_h(r)$ is the average of the densities on that fiber, all at least d by (23). Therefore $\rho_h(r) \geq d$ at every positive depth. Finally (6) implies

$$\mu_h(r) = \frac{3}{2} 3^{-h} \rho_h(r) \geq \frac{3}{2} d 3^{-h}.$$

Taking $c = (3/2)d = (9/2)2^{-V} a > 0$ proves Theorem 1.1.

10 Verification, dependency boundaries, and reproduction

The theorem-specific reverification completed successfully on 12 September 2026. The owning Lake target built successfully, and a separate Lean invocation accepted the complete consolidated appendix. Both commands returned exit code zero. The final theorem's transitive axiom set is exactly

`propext, Classical.choice, Quot.sound.`

No admitted theorem, extra mathematical axiom, or native-evaluation axiom appeared in that footprint. The source scan covered 1542 files, including all 1485 pinned upstream source entries; all manifest hashes and pinned dependency revisions matched. The scanned source hashes were unchanged between the start and finish of the check. This focused verification is recorded in `verification.json`, with `theorem-build.log` and `appendix-lean.log` alongside this document. The SHA-256 of the exact appendix source is

`f6b825def5a01094bc2c7e496cb80ff4a9a995f7aa50f6f444581004436293a7`

The consolidated source in Appendix A contains all five modules of the new comparison-to-floor chain in order. Their local import lines are replaced by three supporting imports at the top. The appended file was compiled separately from the original modules; it is not a hand-transcribed shortened proof. Its last command prints the transitive axioms of the literal theorem.

The library supplies the probability definitions, Fourier mixing, core construction and variation estimates, as well as sibling arithmetic. The appendix therefore compiles *inside the pinned project*, not in an empty Lean installation. It does not reproduce Mathlib or all supporting source files. The complete original supporting library remains in `ProofAtlasAttack/vendor/positive-density-log-time-collatz`. The imported source's corrected release commit is

0aef8b0cfaaf6959500063472f2e328c60df8d54.

From the repository root, reproduce the checks and the document using:

```
cd ProofAtlasAttack
LEAN_NUM_THREADS=4 python3 scripts/verify.py
lake env lean ../output/pdf/syracuse-uniform-floor/SyracuseUniformFloor.lean
cd ../output/pdf/syracuse-uniform-floor
xelatex -halt-on-error syracuse-uniform-floor.tex
xelatex -halt-on-error syracuse-uniform-floor.tex
```

The source is pinned to leanprover/lean4:v4.30.0-rc2. The package check uses cached pinned third-party dependencies; it is not an independent LeanChecker replay or a cold rebuild of all Mathlib. A passing kernel check establishes the exact formal declaration relative to those foundations; it does not establish priority or independently referee this exposition.

The bound is existential and gives no practical numerical constant. It concerns the random variables (2). It does not replace the missing deterministic argument showing that every positive Collatz trajectory reaches 1. In particular, density-one convergence, exclusion of all other cycles, and exclusion of all divergent orbits do not follow merely by restating this probability estimate.

References

- [1] T. Tao, *Almost all orbits of the Collatz map attain almost bounded values*, Forum of Mathematics, Pi 10 (2022), e12. Preprint arXiv:1909.03562. <https://arxiv.org/abs/1909.03562>.
- [2] T. Tao, *Equidistribution of Syracuse random variables and density of Collatz preimages*, 25 January 2020. <https://terrytao.wordpress.com/2020/01/25/equidistribution-of-syracuse-random-variables-and-density-of-collatz-preimages/>.
- [3] L. Mazur, *Explicit Positive-Density Collatz Convergence in Logarithmic Time*, 2026. The local proof uses the pinned, corrected Apache-2.0 Lean source described above. <https://www.proofatlas.ai/formalizations/positive-density-log-time-collatz/>.
- [4] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, CADE 28 (2021), pp. 625–635.
- [5] The mathlib Community, *The Lean mathematical library*, CPP 2020, pp. 367–381. The exact dependency revisions used here are recorded in the accompanying verification report.

A Complete consolidated Lean proof

The following listing is the exact UTF-8 content of `SyracuseUniformFloor.lean`. Automatic line wrapping is typesetting only; use the accompanying source file to run it. The source uses the three declared supporting imports and includes no unfinished proof steps. Comments inherited from early stages describe the scope of those individual modules; the final theorem appears at the end of the complete chain.

```

1 import Erdos1135.ND.PositiveDensity.Geom2ShiftedWideSymmetricCoreSummedVariation
2 import Erdos1135.ND.PositiveDensity.CommonZFullReverseKernel
3 import CollatzTargetDensity.TernarySpread
4
5 /-! Consolidated proof appendix, 2026-09-12.
6 The five development modules below are reproduced without their import lines.
7 Imports above supply the unchanged supporting library and sibling arithmetic. -/
8
9 -- BEGIN SyracuseMinorant.lean
10
11 /-!
12 # One-sided comparison with the actual Syracuse density
13
14 Adapted from Lech Mazur's Apache-2.0 reference-tail transport proof, corrected
15 source commit 0aef8b0cfaaf6959500063472f2e328c60df8d54. Its exact prefix
16 disintegration, Kraft inequality, and Tao mixing theorem are imported unchanged.
17
18 For a lower-bound argument, rejected probability is favorable: the selected
19 part is below the full law. Bounding only the positive part of the difference
20 therefore removes the rejection term from the two-sided estimate. This file
21 does not yet establish an all-depth uniform positive density floor.
22 -/
23
24 namespace CollatzTargetDensity.SyracuseMinorant
25 open Erdos1135 Erdos1135.ND.PositiveDensity
26 open scoped BigOperators
27 noncomputable section
28
29 private theorem mean_abs_sum_le {ι : Type*} [Fintype ι]
30   (q : ℕ) (f : ι → ZMod (3 ^ q) → ℝ) :
31   ndTernaryUniformMean q (fun y => |∑ i, f i y|) ≤
32   ∑ i, ndTernaryUniformMean q (fun y => |f i y|) := by
33   unfold ndTernaryUniformMean
34   rw [← Finset.mul_sum, Finset.sum_comm]
35   apply mul_le_mul_of_nonneg_left _ (by unfold ndTernaryUniformScale; positivity)
36   exact Finset.sum_le_sum fun y _ => Finset.abs_sum_le_sum_abs _ _
37
38 theorem prefix_family_selected_le_density
39   {ι : Type*} [Fintype ι] (q : ℕ) (word : ι → List ℕ+)
40   (hlen : ∀ i, (word i).length ≤ q)
41   (hdisjoint : ∀ i j, i ≠ j → ∀ full : List ℕ+,
42     full.take (word i).length = word i → full.take (word j).length = word j → False)
43   (y : ZMod (3 ^ q)) :
44   (∑ i, ndReferencePrefixTransportAt q (word i) (hlen i)
45     (ndSyracuseUnitReferenceDensity (q - (word i).length)) y) ≤
46     ndSyracuseUnitReferenceDensity q y := by
47   rw [referencePrefixFamily_density_eq_selected q word hlen hdisjoint]
48   let p := Tao.geom2PNatListPMF q
49   let f := Tao.taoSection7OffsetZMod q
50   let E : List ℕ+ → Prop := fun full => ∃ i, full.take (word i).length = word i
51   have hs := Tao.taoGatedSubmass_map_apply_split p E f y
52   have hm : ((p.map f) y).toReal = Tao.syracPMFMassVector q y := by
53   rw [← Tao.syracPMF_eq_geom2PNatListPMF_map_taoSection7OffsetZMod]
54   rfl
55   rw [hm] at hs
56   have hn := Tao.taoGatedSubmass_nonneg p (fun full => ¬ E full) f y
57   have hle : Tao.taoGatedSubmass p E f y ≤ Tao.syracPMFMassVector q y := by
58   linarith
59   have h := mul_le_mul_of_nonneg_left hle
60   (show (0 : ℝ) ≤ (2 / 3 : ℝ) * ((3 ^ q : ℕ) : ℝ) by positivity)

```

```

61   simp only [ndSyracuseUnitReferenceDensity, ndSyracuseUniformDensity, mul_assoc,
62     p, f, E] using h
63
64 /-- The positive-part tail error has no rejected-mass term. -/
65 theorem prefix_family_positive_error_le
66   {ι : Type*} [Fintype ι] (q k : ℕ) (word : ι → List ℕ+)
67   (hlen : ∀ i, (word i).length ≤ q)
68   (hk : ∀ i, k ≤ q - (word i).length)
69   (hdisjoint : ∀ i j, i ≠ j → ∀ full : List ℕ+,
70     full.take (word i).length = word i → full.take (word j).length = word j → False)
71   (eta : ℝ) (heta : 0 ≤ eta)
72   (hmix : ∀ i, Tao.syracFineScaleOscillation k (q - (word i).length) ≤ eta) :
73   ndTernaryUniformMean q (fun y =>
74     max ((∑ i, ndReferencePrefixTransportAt q (word i) (hlen i)
75       (fun z => ndSyracuseUnitReferenceDensity k
76         (Tao.taoZModThreeProjection (hk i) z)) y) -
77       ndSyracuseUnitReferenceDensity q y) 0) ≤ (2 / 3 : ℝ) * eta := by
78   classical
79   let low (i : ι) := ndReferencePrefixTransportAt q (word i) (hlen i)
80     (fun z => ndSyracuseUnitReferenceDensity k (Tao.taoZModThreeProjection (hk i) z))
81   let full (i : ι) := ndReferencePrefixTransportAt q (word i) (hlen i)
82     (ndSyracuseUnitReferenceDensity (q - (word i).length))
83   let atom (i : ι) := (2 : ℝ) ^ (-(Tao.taoTupleWeight (word i) : ℤ))
84   have hedge (i : ι) :
85     ndTernaryUniformMean q (fun y => |low i y - full i y|) ≤
86     atom i * ((2 / 3 : ℝ) * eta) := by
87     simp only [low, full]
88     simp_rw [abs_sub_comm, ← referencePrefixTransportAt_sub]
89     rw [referencePrefixTransportAt_fullL1_eq,
90       unitReferenceDensity_fullL1_eq_twoThirds_oscillation (hk i)]
91     apply mul_le_mul_of_nonneg_left _ (by positivity)
92     exact mul_le_mul_of_nonneg_left (hmix i) (by norm_num)
93   have hsum : (∑ i, atom i) ≤ 1 :=
94     referencePrefixFamily_atom_sum_le_one q word hlen hdisjoint
95   have hreplace :
96     ndTernaryUniformMean q (fun y => |(∑ i, low i y) - ∑ i, full i y|) ≤
97     (2 / 3 : ℝ) * eta := by
98     simp_rw [← Finset.sum_sub_distrib]
99     calc
100       _ ≤ ∑ i, ndTernaryUniformMean q (fun y => |low i y - full i y|) :=
101         mean_abs_sum_le q _
102       _ ≤ ∑ i, atom i * ((2 / 3 : ℝ) * eta) := Finset.sum_le_sum fun i _ => hedge i
103       _ = (∑ i, atom i) * ((2 / 3 : ℝ) * eta) := by rw [Finset.sum_mul]
104       _ ≤ 1 * ((2 / 3 : ℝ) * eta) :=
105         mul_le_mul_of_nonneg_right hsum (mul_nonneg (by norm_num) heta)
106       _ = _ := one_mul _
107   apply le_trans ?_ hreplace
108   unfold ndTernaryUniformMean
109   apply mul_le_mul_of_nonneg_left _ (by unfold ndTernaryUniformScale; positivity)
110   apply Finset.sum_le_sum
111   intro y _
112   have hs : (∑ i, full i y) ≤ ndSyracuseUnitReferenceDensity q y :=
113     prefix_family_selected_le_density q word hlen hdisjoint y
114   apply max_le
115   · have hab := le_abs_self ((∑ i, low i y) - ∑ i, full i y)
116     change (∑ i, low i y) - ndSyracuseUnitReferenceDensity q y ≤ _
117     linarith
118   · exact abs_nonneg _
119
120 theorem prefix_family_positive_error_polynomial (A : ℕ) (hA : 0 < A) :
121   ∃ C : ℝ, 0 ≤ C ∧ ∀ {ι : Type} [Fintype ι] (q k : ℕ)
122   (word : ι → List ℕ+) (hlen : ∀ i, (word i).length ≤ q)
123   (hk : ∀ i, k ≤ q - (word i).length), 1 ≤ k →
124   (∀ i j, i ≠ j → ∀ full : List ℕ+,
125     full.take (word i).length = word i → full.take (word j).length = word j → False) →
126   ndTernaryUniformMean q (fun y =>
127     max ((∑ i, ndReferencePrefixTransportAt q (word i) (hlen i)
128       (fun z => ndSyracuseUnitReferenceDensity k
129         (Tao.taoZModThreeProjection (hk i) z)) y) -
130       ndSyracuseUnitReferenceDensity q y) 0) ≤
131   (2 / 3 : ℝ) * (C / (k : ℝ) ^ A) := by

```

```

132 obtain ⟨C, hC, hmix⟩ := Tao.taoProp114FineScaleMixing A hA
133 refine ⟨C, hC, ?_⟩
134 intro ı _ q k word hlen hk hkpos hdisjoint
135 exact prefix_family_positive_error_le q k word hlen hk hdisjoint
136 (C / (k : ℝ) ^ A) (div_nonneg hC (by positivity))
137 (fun i => hmix _ k hkpos (hk i))
138
139 theorem transport_word_sub (b a K k : ℕ) (f g : ZMod (3 ^ k) → ℝ)
140 (w : ndShiftedReferenceSelectedWords b a K) (y) :
141   ndRootCoreTransportWord b a K k (fun z => f z - g z) w y =
142   ndRootCoreTransportWord b a K k f w y - ndRootCoreTransportWord b a K k g w y := by
143   unfold ndRootCoreTransportWord
144   exact referencePrefixTransportAt_sub _ _ _ _ _
145
146 theorem transport_word_mono (b a K k : ℕ) (f g : ZMod (3 ^ k) → ℝ)
147 (hfg : ∀ z, f z ≤ g z) (w : ndShiftedReferenceSelectedWords b a K) (y) :
148   ndRootCoreTransportWord b a K k f w y ≤ ndRootCoreTransportWord b a K k g w y := by
149   have h := rootCoreTransportWord_nonneg b a K k (fun z => g z - f z)
150   (fun z => sub_nonneg.mpr (hfg z)) w y
151   rw [transport_word_sub] at h
152   linarith
153
154 theorem filtered_kernel_sub (b a K k : ℕ) (psi : List ℕ+ → ℝ)
155 (f g : ZMod (3 ^ k) → ℝ) (y) :
156   ndRootCoreFilteredKernel b a K k psi (fun z => f z - g z) y =
157   ndRootCoreFilteredKernel b a K k psi f y - ndRootCoreFilteredKernel b a K k psi g y := by
158   unfold ndRootCoreFilteredKernel
159   simp_rw [transport_word_sub, mul_sub, Finset.sum_sub_distrib]
160
161 theorem filtered_kernel_mono (b a K k : ℕ) (psi : List ℕ+ → ℝ)
162 (hpsi : ∀ w, 0 ≤ psi w) (f g : ZMod (3 ^ k) → ℝ) (hfg : ∀ z, f z ≤ g z) (y) :
163   ndRootCoreFilteredKernel b a K k psi f y ≤ ndRootCoreFilteredKernel b a K k psi g y := by
164   unfold ndRootCoreFilteredKernel
165   exact Finset.sum_le_sum fun w _ =>
166     mul_le_mul_of_nonneg_left (transport_word_mono b a K k f g hfg w y) (hpsi w.val)
167
168 theorem selected_atom_budget (b a K : ℕ) :
169   (∑ w : ndShiftedReferenceSelectedWords b a K,
170     (2 : ℝ) ^ (-(Tao.taoTupleWeight w.val : ℤ))) ≤ 1 := by
171   apply referencePrefixFamily_atom_sum_le_one (ndGeom2ShiftedWideSymmetricHorizon b)
172   (fun w : ndShiftedReferenceSelectedWords b a K => w.val)
173   (shiftedReferenceSelectedWords_length_le_horizon b a K)
174   exact shiftedReferenceSelectedWords_prefix_disjoint b a K
175
176 /-- Each filtered kernel contracts the mean positive part of a difference. -/
177 theorem filtered_kernel_positive_contraction (b a K k : ℕ) (psi : List ℕ+ → ℝ)
178 (hpsi : ∀ w, 0 ≤ psi w ∧ psi w ≤ 1) (f g : ZMod (3 ^ k) → ℝ) :
179   ndTernaryUniformMean (ndGeom2ShiftedWideSymmetricHorizon b + k) (fun y =>
180     max (ndRootCoreFilteredKernel b a K k psi f y -
181       ndRootCoreFilteredKernel b a K k psi g y) 0) ≤
182     ndTernaryUniformMean k (fun z => max (f z - g z) 0) := by
183   let h : ZMod (3 ^ k) → ℝ := fun z => max (f z - g z) 0
184   have hn : ∀ z, 0 ≤ h z := fun z => le_max_right _ _
185   have hpoint (y) :
186     max (ndRootCoreFilteredKernel b a K k psi f y -
187       ndRootCoreFilteredKernel b a K k psi g y) 0 ≤
188     ndRootCoreFilteredKernel b a K k psi h y := by
189     apply max_le
190     · rw [← filtered_kernel_sub]
191       exact filtered_kernel_mono b a K k psi (fun w => (hpsi w).1) _ h
192       (fun z => le_max_left _ _) y
193     · exact rootCoreFilteredKernel_nonneg b a K k psi (fun w => (hpsi w).1) h hn y
194   have hmean : ndTernaryUniformMean (ndGeom2ShiftedWideSymmetricHorizon b + k) (fun y =>
195     max (ndRootCoreFilteredKernel b a K k psi f y -
196       ndRootCoreFilteredKernel b a K k psi g y) 0) ≤
197     ndTernaryUniformMean (ndGeom2ShiftedWideSymmetricHorizon b + k)
198     (ndRootCoreFilteredKernel b a K k psi h) := by
199     unfold ndTernaryUniformMean
200     exact mul_le_mul_of_nonneg_left (Finset.sum_le_sum fun y _ => hpoint y)
201       (by unfold ndTernaryUniformScale; positivity)
202   have hb : (∑ w : ndShiftedReferenceSelectedWords b a K,

```

```

203     psi w.val * (2 : R) ^ (-(Tao.taoTupleWeight w.val : Z))) ≤ 1 := by
204   apply le_trans ?_ (selected_atom_budget b a K)
205   apply Finset.sum_le_sum
206   intro w _
207   simpa only [one_mul] using
208     mul_le_mul_of_nonneg_right (hpsi w.val).2
209     (by positivity : (0 : R) ≤ (2 : R) ^ (-(Tao.taoTupleWeight w.val : Z)))
210   rw [rootCoreFilteredKernel_fullMean_eq b a K k psi h hn] at hmean
211   have hh : 0 ≤ ndTernaryUniformMean k h := by
212     unfold ndTernaryUniformMean ndTernaryUniformScale
213     exact mul_nonneg (by positivity) (Finset.sum_nonneg fun z _ => hn z)
214   exact hmean.trans (by simpa only [one_mul] using mul_le_mul_of_nonneg_right hb hh)
215
216 /-- One-stage upper error against the full law, uniformly over all root-side
217 parameters. No strip rejection estimate or assumption about convergence is used. -/
218 theorem filtered_kernel_positive_error_polynomial (A : ℕ) (hA : 0 < A) :
219   ∃ C : ℝ, 0 ≤ C ∧ ∀ b a K k : ℕ, 1 ≤ k → ∀ psi : List ℕ+ → ℝ,
220     (∀ w, psi w ≤ 1) →
221     ndTernaryUniformMean (ndGeom2ShiftedWideSymmetricHorizon b + k) (fun y =>
222       max (ndRootCoreFilteredKernel b a K k psi (ndSyracuseUnitReferenceDensity k) y -
223         ndSyracuseUnitReferenceDensity (ndGeom2ShiftedWideSymmetricHorizon b + k) y) 0) ≤
224       C / (k : ℝ) ^ A := by
225   obtain ⟨C, hC, hrate⟩ := prefix_family_positive_error_polynomial A hA
226   refine ⟨(2 / 3 : ℝ) * C, by positivity, ?_⟩
227   intro b a K k hk psi hpsi
228   let q := ndGeom2ShiftedWideSymmetricHorizon b + k
229   let word := fun w : ndShiftedReferenceSelectedWords b a K => w.val
230   have hlen (w : ndShiftedReferenceSelectedWords b a K) : (word w).length ≤ q := by
231     have := shiftedReferenceSelectedWords_length_le_horizon b a K w
232     dsimp [word, q]; omega
233   have ht (w : ndShiftedReferenceSelectedWords b a K) : k ≤ q - (word w).length := by
234     have := shiftedReferenceSelectedWords_length_le_horizon b a K w
235     dsimp [word, q]; omega
236   have hr := hrate q k word hlen ht hk (shiftedReferenceSelectedWords_prefix_disjoint b a K)
237   have hp (y : ZMod (3 ^ q)) :
238     ndRootCoreFilteredKernel b a K k psi (ndSyracuseUnitReferenceDensity k) y ≤
239     ∑ w : ndShiftedReferenceSelectedWords b a K,
240     ndRootCoreTransportWord b a K k (ndSyracuseUnitReferenceDensity k) w y := by
241     unfold ndRootCoreFilteredKernel
242     apply Finset.sum_le_sum
243     intro w _
244     simpa only [one_mul] using mul_le_mul_of_nonneg_right (hpsi w.val)
245     (rootCoreTransportWord_nonneg b a K k (ndSyracuseUnitReferenceDensity k)
246       (ndSyracuseUnitReferenceDensity_nonneg k) w y)
247   have hmean :
248     ndTernaryUniformMean q (fun y =>
249       max (ndRootCoreFilteredKernel b a K k psi (ndSyracuseUnitReferenceDensity k) y -
250         ndSyracuseUnitReferenceDensity q y) 0) ≤
251     ndTernaryUniformMean q (fun y =>
252       max ((∑ w : ndShiftedReferenceSelectedWords b a K,
253         ndRootCoreTransportWord b a K k (ndSyracuseUnitReferenceDensity k) w y) -
254         ndSyracuseUnitReferenceDensity q y) 0) := by
255     unfold ndTernaryUniformMean
256     apply mul_le_mul_of_nonneg_left _ (by unfold ndTernaryUniformScale; positivity)
257     exact Finset.sum_le_sum fun y _ => max_le_max (sub_le_sub_right (hp y) _) le_rfl
258   have hresult := hmean.trans hr
259   simpa only [mul_div_assoc] using hresult
260
261 end
262 end CollatzTargetDensity.SyracuseMinorant
263 -- END SyracuseMinorant.lean
264
265 -- BEGIN SyracuseCoreMinorant.lean
266
267 /-!
268 # All-generation one-sided core comparison
269
270 This compares the literal imported backward core marks with the actual
271 finite Syracuse densities. No positivity of the limiting core or uniform
272 Syracuse floor is assumed or asserted here.
273 -/

```

```

274
275 namespace CollatzTargetDensity.SyracuseMinorant
276 open Erdos1135 Erdos1135.ND.PositiveDensity
277 open scoped BigOperators
278 noncomputable section
279
280 theorem mean_positive_triangle (q : ℕ) (f g h : ZMod (3 ^ q) → ℝ) :
281   ndTernaryUniformMean q (fun y => max (f y - h y) 0) ≤
282     ndTernaryUniformMean q (fun y => max (f y - g y) 0) +
283     ndTernaryUniformMean q (fun y => max (g y - h y) 0) := by
284   unfold ndTernaryUniformMean
285   rw [← mul_add, ← Finset.sum_add_distrib]
286   apply mul_le_mul_of_nonneg_left _ (by unfold ndTernaryUniformScale; positivity)
287   apply Finset.sum_le_sum
288   intro y _
289   apply max_le
290   · have h1 := le_max_left (f y - g y) 0
291     have h2 := le_max_left (g y - h y) 0
292     linarith
293   · exact add_nonneg (le_max_right _ _) (le_max_right _ _)
294
295 theorem core_word_filter_bounds (b m : ℕ) (w : List ℕ+) :
296   0 ≤ ndRootCoreWordFilter b m w ∧ ndRootCoreWordFilter b m w ≤ 1 := by
297   unfold ndRootCoreWordFilter
298   split_ifs <;> norm_num
299
300 /-- The error depends on generation and terminal conductor, but not on the
301 root, initial floor, cap schedule, strip widths, or backward horizon. -/
302 theorem backward_mark_positive_error_polynomial (A : ℕ) (hA : 0 < A) :
303   ∃ C : ℝ, 0 ≤ C ∧ ∀ (b : ℕ) (cap width : ℕ → ℕ) (k n : ℕ), 1 ≤ k →
304     ndTernaryUniformMean (ndRootCoreBackwardConductor b k n) (fun y =>
305       max (ndRootCoreBackwardMark b cap width k n y -
306         ndSyracuseUnitReferenceDensity (ndRootCoreBackwardConductor b k n) y) 0) ≤
307       (n : ℝ) * (C / (k : ℝ) ^ A) := by
308   obtain ⟨C, hC, hrate⟩ := filtered_kernel_positive_error_polynomial A hA
309   refine ⟨C, hC, ?_⟩
310   intro b cap width k n hk
311   induction n generalizing b cap with
312   | zero =>
313     simp [ndRootCoreBackwardConductor, ndRootCoreBackwardMark,
314       ndTernaryUniformMean]
315   | succ n ih =>
316     let b' := b + b / 100
317     let q := ndRootCoreBackwardConductor b' k n
318     let tail := ndRootCoreBackwardMark b' (ndGeom2RootSideCapTail cap) width k n
319     let psi := ndRootCoreWordFilter b (width b)
320     let K := ndRootCoreFilteredKernel b (ndGeom2ShiftedWideSymmetricShiftRadius b) (cap 0) q psi
321     have hq : k ≤ q := rootCoreBackwardConductor_ge b' k n
322     have hqpos : 1 ≤ q := hk.trans hq
323     have hp : ∀ w, 0 ≤ psi w ∧ psi w ≤ 1 := core_word_filter_bounds b (width b)
324     have hcontract := filtered_kernel_positive_contraction b
325       (ndGeom2ShiftedWideSymmetricShiftRadius b) (cap 0) q psi hp
326     tail (ndSyracuseUnitReferenceDensity q)
327     have hstep := hrate b (ndGeom2ShiftedWideSymmetricShiftRadius b) (cap 0) q hqpos psi
328     (fun w => (hp w).2)
329     have htri := mean_positive_triangle (ndGeom2ShiftedWideSymmetricHorizon b + q)
330       (K tail) (K (ndSyracuseUnitReferenceDensity q))
331       (ndSyracuseUnitReferenceDensity (ndGeom2ShiftedWideSymmetricHorizon b + q))
332     have hprev := ih b' (ndGeom2RootSideCapTail cap)
333     have hden : C / (q : ℝ) ^ A ≤ C / (k : ℝ) ^ A := by
334       apply div_le_div_of_nonneg_left hC (by positivity)
335     exact pow_le_pow_left (by positivity) (by exact_mod_cast hq) A
336   change ndTernaryUniformMean (ndGeom2ShiftedWideSymmetricHorizon b + q) (fun y =>
337     max (K tail y - ndSyracuseUnitReferenceDensity
338       (ndGeom2ShiftedWideSymmetricHorizon b + q) y) 0) ≤ _
339   change ndTernaryUniformMean q (fun y =>
340     max (tail y - ndSyracuseUnitReferenceDensity q y) 0) ≤ _ at hprev
341   change ndTernaryUniformMean (ndGeom2ShiftedWideSymmetricHorizon b + q) (fun y =>
342     max (K tail y - K (ndSyracuseUnitReferenceDensity q) y) 0) ≤ _ at hcontract
343   change ndTernaryUniformMean (ndGeom2ShiftedWideSymmetricHorizon b + q) (fun y =>
344     max (K (ndSyracuseUnitReferenceDensity q) y -

```

```

345     ndSyracuseUnitReferenceDensity (ndGeom2ShiftedWideSymmetricHorizon b + q) y) 0) ≤ _ at hstep
346   push_cast
347   nlinarith
348
349 end
350 end CollatzTargetDensity.SyracuseMinorant
351 -- END SyracuseCoreMinorant.lean
352
353 -- BEGIN SyracusePositiveCylinder.lean
354
355 /-!
356 # A fixed cylinder of positive backward core marks
357
358 The floor schedule is made independent of physical representatives. Mazur's
359 root-uniform variation bound is then applied after freezing a finite cylinder.
360 -/
361
362 namespace CollatzTargetDensity.SyracuseMinorant
363 open Erdos1135 Erdos1135.ND.PositiveDensity Filter
364 open scoped Topology
365 noncomputable section
366
367 def coreFloor (b : ℕ) : ℕ → ℕ
368   | 0 => b
369   | n + 1 => coreFloor (b + b / 100) n
370
371 def terminalConductor (b n : ℕ) : ℕ := coreFloor b n / 4
372
373 def ambientConductor (b n : ℕ) : ℕ :=
374   ndRootCoreBackwardConductor b (terminalConductor b n) n
375
376 def coreMark (b n : ℕ) : ZMod (3 ^ ambientConductor b n) → ℝ :=
377   ndRootCoreBackwardMark b (fun n => n / 100) ndRootCoreWidth (terminalConductor b n) n
378
379 theorem forward_floor_eq_coreFloor
380   (U : NDGeom2ShiftedWideSymmetricRootSideUniformFloorState)
381   (cap : ℕ → ℕ) (n : ℕ) :
382   (U.forwardIterate cap n).floor = coreFloor U.floor n := by
383   induction n generalizing U cap with
384   | zero => rfl
385   | succ n ih =>
386     change ((U.next (cap 0)).forwardIterate (ndGeom2RootSideCapTail cap) n).floor = _
387     rw [ih]
388     rfl
389
390 theorem coreFloor_linear_lower {b : ℕ} (hb : 200 ≤ b) (n : ℕ) :
391   b + 2 * n ≤ coreFloor b n := by
392   induction n generalizing b with
393   | zero => simp [coreFloor]
394   | succ n ih =>
395     have hdiv : 2 ≤ b / 100 := by omega
396     have h := ih (b := b + b / 100) (by omega)
397     simp only [coreFloor]
398     omega
399
400 theorem terminalConductor_pos {b : ℕ} (hb : 200 ≤ b) (n : ℕ) :
401   1 ≤ terminalConductor b n := by
402   have h := coreFloor_linear_lower hb n
403   unfold terminalConductor
404   omega
405
406 theorem generation_le_twice_terminal {b : ℕ} (hb : 200 ≤ b) (n : ℕ) :
407   n ≤ 2 * terminalConductor b n := by
408   have h := coreFloor_linear_lower hb n
409   unfold terminalConductor
410   omega
411
412 theorem terminal_le_ambient (b n : ℕ) :
413   terminalConductor b n ≤ ambientConductor b n :=
414   rootCoreBackwardConductor_ge b (terminalConductor b n) n
415

```

```

416 theorem singleton_sequence_eq_coreMark (b M : ℕ) (hb : 200 ≤ b)
417   (hM : Odd M) (hl : 16 ^ b ≤ M) (n : ℕ) :
418   (ndRootCoreSingletonState b M hb hM hl).coreMarkedSequence
419     (fun n => n / 100) ndRootCoreWidth n =
420     coreMark b n (M : ZMod (3 ^ ambientConductor b n)) := by
421   unfold NDGeom2ShiftedWideSymmetricRootSideUniformFloorState.coreMarkedSequence
422   rw [forward_floor_eq_coreFloor]
423   exact rootCoreSingletonState_markedMass_eq_backwardMark b M hb hM hl
424     (fun n => n / 100) ndRootCoreWidth n (terminalConductor b n)
425     (terminalConductor_pos hb n)
426
427 /-- One cylinder and one positive margin work for all later generations and
428 all eligible odd physical representatives of that cylinder. -/
429 theorem exists_fixed_positive_core_cylinder {b : ℕ} (hb : 32 ^ 5 ≤ b) :
430   ∃ (q N : ℕ) (y : ZMod (3 ^ q)) (a : ℝ),
431     1 ≤ q ∧ IsUnit (y.val : ZMod (3 ^ 1)) ∧ 0 < a ∧
432     ∀ j, N ≤ j → ∀ M : ℕ, Odd M → 16 ^ b ≤ M →
433       (M : ZMod (3 ^ q)) = y →
434       a ≤ coreMark b j (M : ZMod (3 ^ ambientConductor b j)) := by
435   let cap : ℕ → ℕ := fun n => n / 100
436   have hb200 : 200 ≤ b := by omega
437   have hwidth (B : ℕ) (hB : b ≤ B) : 2 ≤ ndRootCoreWidth B := by
438     have h := (rootCoreWidth_guard (hb.trans hB)).1
439     omega
440   have hupper (n : ℕ) : cap n ≤ 1 * (n + 1) := by
441     dsimp [cap]
442     have := Nat.div_le_self n 100
443     omega
444   have hlower (n : ℕ) : 0 + n / 100 ≤ cap n := by simp [cap]
445   obtain ⟨M0, hM0, hL0, _, _, _, _⟩ :=
446     exists_successful_core_singleton_at_generation hb200 (by norm_num : 1 ≤ 1)
447     cap ndRootCoreWidth hwidth 0 0 (by norm_num : (1 : ℝ) ≤ 1)
448   let U0 := ndRootCoreSingletonState b M0 hb200 hM0 hL0
449   obtain ⟨p, hp, hprod⟩ := U0.exists_pos_le_coreProbabilityProduct hb cap 0 hlower
450   change ∀ n, p ≤ ndRootCoreProbabilityProduct b cap ndRootCoreWidth n at hprod
451   obtain ⟨C, _, hlimall⟩ :=
452     NDGeom2ShiftedWideSymmetricRootSideUniformFloorState.exists_coreMarkedLimit_with_root_uniform_tail
453   have ht : ∀f N in atTop, ndRootCoreVariationTail b 1 0 C N < p / 6 :=
454     (tendsto_order.mp (tendsto_rootCoreVariationTail_zero b 1 0 C)).2 (p / 6) (by positivity)
455   obtain ⟨N, hN⟩ := Filter.eventually_atTop.mp ht
456   obtain ⟨y, hyunit, hy⟩ := exists_unit_rootCoreBackwardMark_ge_product
457     (by omega : 9 ≤ b) (terminalConductor_pos hb200 N) cap ndRootCoreWidth hwidth N
458   refine ⟨ambientConductor b N, N, y, p / 3,
459     (terminalConductor_pos hb200 N).trans (terminal_le_ambient b N), hyunit,
460     by positivity, ?_⟩
461   intro j hj M hM hl hMy
462   let U := ndRootCoreSingletonState b M hb200 hM hl
463   obtain ⟨ell, _, _, htail⟩ := hlimall U hb cap 1 0 hupper hlower
464   have hD : U.denominator = 1 := rootCoreSingletonState_denominator_eq_one b M hb200 hM hl
465   have hdevN := htail N
466   have hdevj := htail j
467   rw [hD, one_mul] at hdevN hdevj
468   change |ell - U.coreMarkedSequence cap ndRootCoreWidth N| ≤
469     ndRootCoreVariationTail b 1 0 C N at hdevN
470   change |ell - U.coreMarkedSequence cap ndRootCoreWidth j| ≤
471     ndRootCoreVariationTail b 1 0 C j at hdevj
472   have heq (i : ℕ) : U.coreMarkedSequence cap ndRootCoreWidth i =
473     coreMark b i (M : ZMod (3 ^ ambientConductor b i)) :=
474     singleton_sequence_eq_coreMark b M hb200 hM hl i
475   rw [heq N, hMy] at hdevN
476   rw [heq j] at hdevj
477   change (2 / 3 : ℝ) * ndRootCoreProbabilityProduct b cap ndRootCoreWidth N ≤
478     coreMark b N y at hy
479   have hn := hN N le_rfl
480   have hj' := hN j hj
481   have hpn := hprod N
482   have hdN := (abs_le.mp hdevN).1
483   have hdj := (abs_le.mp hdevj).2
484   linarith
485
486 /-- Arbitrarily large odd representatives allow the physical-root statement

```

```

487 to be used at every atom of every sufficiently deep finite quotient. -/
488 theorem exists_fixed_positive_core_cylinder_finite {b : ℕ} (hb : 32 ^ 5 ≤ b) :
489   ∃ (q N : ℕ) (y : ZMod (3 ^ q)) (a : ℝ),
490     1 ≤ q ∧ IsUnit (y.val : ZMod (3 ^ 1)) ∧ 0 < a ∧
491     ∀ j, N ≤ j → ∀ hq : q ≤ ambientConductor b j,
492     ∀ x : ZMod (3 ^ ambientConductor b j),
493       Tao.taoZModThreeProjection hq x = y → a ≤ coreMark b j x := by
494   obtain ⟨q, N, y, a, hq, hy, ha, hmark⟩ := exists_fixed_positive_core_cylinder hb
495   refine ⟨q, N, y, a, hq, hy, ha, ?_⟩
496   intro j hj hqj x hx
497   obtain ⟨M, hM, hm, _, hres⟩ := exists_large_logTimeSuccessful_root_in_residue
498     (ambientConductor b j) (16 ^ b) ⟨x.val, x.val_lt⟩ (by norm_num : (1 : ℝ) ≤ 1)
499   have hcast : (M : ZMod (3 ^ ambientConductor b j)) = x := by
500     apply ZMod.val_injective
501     simpa only [ZMod.val_natCast] using hres
502   have hcastq : (M : ZMod (3 ^ q)) = y := by
503     rw [← Tao.taoZModThreeProjection_natCast hqj M, hcast, hx]
504   have h := hmark j hj M hm.2.1 ((le_max_left _ _).trans hm.le) hcastq
505   rwa [hcast] at h
506
507 end
508 end CollatzTargetDensity.SyracuseMinorant
509 -- END SyracusePositiveCylinder.lean
510
511 -- BEGIN SyracuseCylinderProjection.lean
512
513 /-! Fixed-cylinder projection of the one-sided comparison. -/
514 namespace CollatzTargetDensity.SyracuseMinorant
515 open Erdos1135 Erdos1135.ND.PositiveDensity Filter
516 open scoped BigOperators Topology
517 noncomputable section
518
519 theorem projection_fiber_const {h q : ℕ} (hhq : h ≤ q)
520   (x : ZMod (3 ^ h)) (a : ℝ) :
521   (∑ y : ZMod (3 ^ q), if Tao.taoZModThreeProjection hhq y = x then a else 0) =
522   (3 : ℝ) ^ (q - h) * a := by
523   classical
524   rw [← Finset.sum_filter]
525   rw [Finset.sum_subtype _ (by simp : ∀ y : ZMod (3 ^ q),
526     y ∈ Finset.univ.filter (fun y => Tao.taoZModThreeProjection hhq y = x) ↔
527     Tao.taoZModThreeProjection hhq y = x)]
528   have hc : Fintype.card {y : ZMod (3 ^ q) // Tao.taoZModThreeProjection hhq y = x} =
529     3 ^ (q - h) := Tao.taoZModThreeProjection_fiber_card hhq x
530   rw [Finset.sum_const, Finset.card_univ, hc]
531   simp
532
533 theorem projection_fiber_density {h q : ℕ} (hhq : h ≤ q)
534   (x : ZMod (3 ^ h)) :
535   (∑ y : ZMod (3 ^ q), if Tao.taoZModThreeProjection hhq y = x then
536     ndSyracuseUnitReferenceDensity q y else 0) =
537   (3 : ℝ) ^ (q - h) * ndSyracuseUnitReferenceDensity h x := by
538   classical
539   have hmass : (∑ y : ZMod (3 ^ q), if Tao.taoZModThreeProjection hhq y = x then
540     (Tao.syracPMF q y).toReal else 0) = (Tao.syracPMF h x).toReal := by
541     have he := congrArg (fun p => (p x).toReal)
542     (Tao.syracPMF_map_taoZModThreeProjection_eq_of_le hhq)
543     dsimp only at he
544     rw [Tao.pmf_map_apply_toReal_tsum, tsum_fintype] at he
545     simpa only [eq_comm] using he
546   unfold ndSyracuseUnitReferenceDensity ndSyracuseUniformDensity Tao.syracPMFMassVector
547   simp_rw [← mul_assoc]
548   rw [show (∑ y : ZMod (3 ^ q), if Tao.taoZModThreeProjection hhq y = x then
549     (2 / 3 : ℝ) * (3 ^ q : ℕ) * (Tao.syracPMF q y).toReal else 0) =
550     (2 / 3 : ℝ) * (3 ^ q : ℕ) * ∑ y : ZMod (3 ^ q),
551     if Tao.taoZModThreeProjection hhq y = x then (Tao.syracPMF q y).toReal else 0 by
552     rw [Finset.mul_sum]; apply Finset.sum_congr rfl; intro y _; split_ifs <=> simp]
553   rw [hmass]
554   push_cast
555   have hp : (3 : ℝ) ^ q = 3 ^ (q - h) * 3 ^ h := by
556     rw [← pow_add, Nat.sub_add_cancel hhq]
557   rw [hp]

```

```

558   ring
559
560   /-- The test conductor h is fixed; the factor is 3^h, not 3^q. -/
561   theorem fiber_lower_of_positive_error {h q : ℕ} (hhq : h ≤ q)
562     (x : ZMod (3 ^ h)) (g : ZMod (3 ^ q) → ℝ) (a : ℝ)
563     (hg : ∀ y, Tao.taoZModThreeProjection hhq y = x → a ≤ g y) :
564     a ≤ ndSyracuseUnitReferenceDensity h x + (3 : ℝ) ^ h *
565       ndTernaryUniformMean q (fun y => max (g y - ndSyracuseUnitReferenceDensity q y) 0) := by
566     classical
567     have hsum : (∑ y : ZMod (3 ^ q), if Tao.taoZModThreeProjection hhq y = x then a else 0) ≤
568       (∑ y : ZMod (3 ^ q), if Tao.taoZModThreeProjection hhq y = x then
569         ndSyracuseUnitReferenceDensity q y else 0) +
570       ∑ y : ZMod (3 ^ q), max (g y - ndSyracuseUnitReferenceDensity q y) 0 := by
571       rw [← Finset.sum_add_distrib]
572       apply Finset.sum_le_sum
573       intro y _
574       by_cases hy : Tao.taoZModThreeProjection hhq y = x
575       · simp only [if_pos hy]
576         have := hg y hy
577         have := le_max_left (g y - ndSyracuseUnitReferenceDensity q y) 0
578         linarith
579       · simp only [if_neg hy, zero_add]
580         exact le_max_right _ _
581     rw [projection_fiber_const, projection_fiber_density] at hsum
582     have hp : (3 : ℝ) ^ q = 3 ^ (q - h) * 3 ^ h := by
583       rw [← pow_add, Nat.sub_add_cancel hhq]
584     unfold ndTernaryUniformMean ndTernaryUniformScale
585     push_cast
586     apply le_of_mul_le_mul_left _ (show (0 : ℝ) < 3 ^ (q - h) by positivity)
587     calc
588       _ ≤ _ := hsum
589       _ = _ := by rw [hp]; field_simp
590
591   theorem coreMark_positive_error_rate {b : ℕ} (hb : 200 ≤ b) :
592     ∃ C : ℝ, 0 ≤ C ∧ ∀ n : ℕ, 0 < n →
593       ndTernaryUniformMean (ambientConductor b n) (fun y =>
594         max (coreMark b n y - ndSyracuseUnitReferenceDensity (ambientConductor b n) y) 0) ≤
595       C / (n : ℝ) := by
596     obtain ⟨C, hC, hr⟩ := backward_mark_positive_error_polynomial 2 (by norm_num)
597     refine ⟨4 * C, by positivity, ?_⟩
598     intro n hn
599     have hk := terminalConductor_pos hb n
600     have hnk := generation_le_twice_terminal hb n
601     have hbase := hr b (fun n => n / 100) ndRootCoreWidth
602       (terminalConductor b n) n hk
603     refine hbase.trans ?_
604     have hkR : 0 < (terminalConductor b n : ℝ) := by exact_mod_cast hk
605     have hnR : 0 < (n : ℝ) := by exact_mod_cast hn
606     have hnkR : (n : ℝ) ≤ 2 * (terminalConductor b n : ℝ) := by exact_mod_cast hnk
607     apply (le_div_iff₀ hnkR).2
608     rw [← mul_div_assoc, div_mul_eq_mul_div]
609     apply (div_le_iff₀ (sq_pos_of_pos hkR)).2
610     have hs : (n : ℝ) ^ 2 ≤ 4 * (terminalConductor b n : ℝ) ^ 2 := by nlinarith
611     have := mul_le_mul_of_nonneg_left hs hC
612     nlinarith
613
614   /-- The actual Syracuse densities are bounded below on every finer subcell
615   of one fixed positive unit cylinder. No moving-conductor limit is used. -/
616   theorem exists_positive_syracuse_cylinder :
617     ∃ (q : ℕ) (y : ZMod (3 ^ q)) (a : ℝ),
618       1 ≤ q ∧ IsUnit (y.val : ZMod (3 ^ 1)) ∧ 0 < a ∧
619       ∀ h, ∀ hqh : q ≤ h, ∀ x : ZMod (3 ^ h),
620         Tao.taoZModThreeProjection hqh x = y →
621         a ≤ ndSyracuseUnitReferenceDensity h x := by
622     let b : ℕ := 32 ^ 5
623     have hb : 32 ^ 5 ≤ b := le_refl
624     obtain ⟨q, N, y, a, hq, hy, ha, hmark⟩ := exists_fixed_positive_core_cylinder_finite hb
625     obtain ⟨C, hC, herr⟩ := coreMark_positive_error_rate (b := b) (by norm_num [b])
626     refine ⟨q, y, a, hq, hy, ha, ?_⟩
627     intro h hqh x hx
628     by_contra hnot

```

```

629   have hdelta : 0 < a - ndSyracuseUnitReferenceDensity h x := by linarith
630   obtain ⟨n, hn⟩ := exists_nat_gt (max ((N + 2 * h + 1 : ℕ) : ℝ)
631     (C * (3 : ℝ) ^ h / (a - ndSyracuseUnitReferenceDensity h x)))
632   have hlarge : N + 2 * h + 1 < n := by
633     exact_mod_cast (lt_of_le_of_lt (le_max_left _ _) hn)
634   have hratio : C * (3 : ℝ) ^ h / (a - ndSyracuseUnitReferenceDensity h x) < n :=
635     lt_of_le_of_lt (le_max_right _ _) hn
636   have hn0 : 0 < n := by omega
637   have hhn : h ≤ ambientConductor b n := by
638     have := generation_le_twice_terminal (b := b) (by norm_num [b]) n
639     have := terminal_le_ambient b n
640     omega
641   have hpoin := fiber_lower_of_positive_error hhn x (coreMark b n) a (by
642     intro z hz
643     apply hmark n (by omega) (hqh.trans hhn) z
644     rw [← Tao.taoZModThreeProjection_comp_apply hqh hhn, hz, hx])
645   have herror := herr n hn0
646   have hupper : a ≤ ndSyracuseUnitReferenceDensity h x + (3 : ℝ) ^ h * (C / (n : ℝ)) :=
647     hpoin.trans (by gcongr)
648   have hnR : 0 < (n : ℝ) := by exact_mod_cast hn0
649   have hstrict : (3 : ℝ) ^ h * (C / (n : ℝ)) < a - ndSyracuseUnitReferenceDensity h x := by
650     have hh := (div_lt_iff₀ hdelta).mp hratio
651     rw [← mul_div_assoc]
652     apply (div_lt_iff₀ hnR).2
653     nlinarith
654   linarith
655
656 end
657 end CollatzTargetDensity.SyracuseMinorant
658 -- END SyracuseCylinderProjection.lean
659
660 -- BEGIN SyracuseCylinderSpread.lean
661
662 /-! Bounded inverse coverage and propagation of a positive Syracuse cylinder. -/
663 namespace CollatzTargetDensity.SyracuseMinorant
664 open Erdos1135 Erdos1135.ND.PositiveDensity
665 open scoped BigOperators
666 noncomputable section
667
668 theorem syracStep_natCast_of_equation (h c : ℕ) (v : ℕ+)
669   (r : ZMod (3 ^ (h + 1))) (he : 3 * c + 1 = 2 ^ (v : ℕ) * r.val) :
670   Tao.syracStep h (c : ZMod (3 ^ h)) v = r := by
671   have hm : 3 * (c % 3 ^ h) + 1 ≡ 3 * c + 1 [MOD 3 ^ (h + 1)] := by
672     simp only [pow_succ, Nat.mul_comm] using
673       ((Nat.mod_modEq c (3 ^ h)).mul_left' 3).add_right 1
674   rw [he] at hm
675   have hc := (ZMod.natCast_eq_natCast_iff _ _).mpr hm
676   have hinner : (3 : ZMod (3 ^ (h + 1))) * (c % 3 ^ h : ℕ) + 1 =
677     (2 : ZMod (3 ^ (h + 1))) ^ (v : ℕ) * r := by
678     simp only [Nat.cast_add, Nat.cast_mul, Nat.cast_pow, Nat.cast_ofNat,
679       Nat.cast_one, ZMod.natCast_zmod_val] using hc
680   unfold Tao.syracStep
681   rw [ZMod.val_natCast, hinner, ← mul_assoc,
682     Tao.taoZModThreePow_inv_two_pow_mul, one_mul]
683
684 /- Every unit target has a one-step inverse in a prescribed q-cylinder,
685 with valuation bounded in terms of q alone, independently of target depth. -/
686 theorem bounded_inverse_into_cylinder (q h : ℕ) (hqh : q ≤ h)
687   (y : ZMod (3 ^ q)) (r : ZMod (3 ^ (h + 1))) (hr : r.val % 3 ≠ 0) :
688   ∃ (v : ℕ+) (z : ZMod (3 ^ h)),
689     (v : ℕ) ≤ 2 * 3 ^ q + 2 ∧
690     Tao.taoZModThreeProjection hqh z = y ∧ Tao.syracStep h z v = r := by
691   have hbase : ∃ c e : ℕ, 1 ≤ e ∧ e ≤ 2 ∧ 3 * c + 1 = 2 ^ e * r.val := by
692     by_cases hr1 : r.val % 3 = 1
693     · refine ⟨(4 * r.val - 1) / 3, 2, by omega, by omega, ?_⟩
694       norm_num
695       omega
696     · refine ⟨(2 * r.val - 1) / 3, 1, by omega, by omega, ?_⟩
697       norm_num
698       omega
699   obtain ⟨c, e, he0, he2, he⟩ := hbase

```

```

700 obtain ⟨t, ht, hres⟩ := sibling_residue q c ⟨y.val, y.val_lt⟩
701 let v : ℕ+ := ⟨2 * t + e, by omega⟩
702 let z : ZMod (3 ^ h) := sibling t c
703 refine ⟨v, z, by dsimp [v]; omega, ?_, ?_⟩
704 · change Tao.taoZModThreeProjection hqh (sibling t c : ZMod (3 ^ h)) = y
705   rw [Tao.taoZModThreeProjection_natCast]
706   apply ZMod.val_injective
707   simpa only [ZMod.val_natCast] using hres
708 · apply syracStep_natCast_of_equation h (sibling t c) v r
709   have hsource := three_mul_ndA5PreviousPhysicalCollapseSource_add_one t
710   have hp : (2 : ℕ) ^ (2 * t) = 4 ^ t := by rw [pow_mul]; norm_num
711   change 3 * sibling t c + 1 = 2 ^ (2 * t + e) * r.val
712   rw [pow_add, hp, mul_assoc, ← he]
713   unfold sibling
714   nlinarith
715
716 theorem syracPMF_single_reverse_le (h : ℕ) (r : ZMod (3 ^ (h + 1)))
717   (z : ZMod (3 ^ h)) (v : ℕ+) (he : Tao.syracStep h z v = r) :
718   (Tao.syracPMF h z).toReal * (1 / 2 : ℝ) ^ (v : ℕ) ≤
719   (Tao.syracPMF (h + 1) r).toReal := by
720   classical
721   have hs : Summable (fun a => ndSyracuseFullReverseLabelMass h r z a) := by
722     simpa using summable_ndSyracuseFullReverseLabelMass h r.val z
723   have hsingle := hs.le_tsum v (fun a _ => ndSyracuseFullReverseLabelMass_nonneg h r z a)
724   have houter := Finset.single_le_sum
725     (f := fun y => ∑' a : ℕ+, ndSyracuseFullReverseLabelMass h r y a)
726     (fun y _ => tsum_nonneg (fun a => ndSyracuseFullReverseLabelMass_nonneg h r y a))
727     (Finset.mem_univ z)
728   rw [sum_ndSyracuseFullReverseLabelMass_eq_syracPMF] at houter
729   have hh := hsingle.trans houter
730   simpa only [ndSyracuseFullReverseLabelMass, if_pos he, Tao.geom2PNat_apply_toReal] using hh
731
732 theorem referenceDensity_single_reverse_le (h : ℕ) (r : ZMod (3 ^ (h + 1)))
733   (z : ZMod (3 ^ h)) (v : ℕ+) (he : Tao.syracStep h z v = r) :
734   3 * (1 / 2 : ℝ) ^ (v : ℕ) * ndSyracuseUnitReferenceDensity h z ≤
735   ndSyracuseUnitReferenceDensity (h + 1) r := by
736   have hh := mul_le_mul_of_nonneg_left (syracPMF_single_reverse_le h r z v he)
737     (show (0 : ℝ) ≤ (2 / 3) * 3 ^ (h + 1) by positivity)
738   unfold ndSyracuseUnitReferenceDensity ndSyracuseUniformDensity Tao.syracPMFMassVector
739   push_cast
740   simpa only [pow_succ] using (by convert hh using 1 <=> ring)
741
742 theorem unit_residue_of_projection {h H : ℕ} (hh : 1 ≤ h) (hle : h ≤ H)
743   (z : ZMod (3 ^ H)) (x : ZMod (3 ^ h))
744   (he : Tao.taoZModThreeProjection hle z = x) (hx : x.val % 3 ≠ 0) :
745   z.val % 3 ≠ 0 := by
746   have hv := congrArg ZMod.val he
747   rw [Tao.taoZModThreeProjection_eq_natCast_val, ZMod.val_natCast] at hv
748   rw [← hv, Nat.mod_mod_of_dvd _ (dvd_pow_self 3 (by omega : h ≠ 0))] at hx
749   exact hx
750
751 /-- A depth-independent lower bound for the actual normalized Syracuse
752 density on every unit residue. -/
753 theorem exists_uniform_positive_syracuse_density :
754   ∃ c : ℝ, 0 < c ∧ ∀ h : ℕ, 1 ≤ h → ∀ r : ZMod (3 ^ h),
755   r.val % 3 ≠ 0 → c ≤ ndSyracuseUnitReferenceDensity h r := by
756   obtain ⟨q, y, a, hq, _, ha, hfloor⟩ := exists_positive_syracuse_cylinder
757   let V : ℕ := 2 * 3 ^ q + 2
758   let c : ℝ := 3 * (1 / 2 : ℝ) ^ V * a
759   have hc : 0 < c := by dsimp [c]; positivity
760   have hlarge : ∀ h : ℕ, q + 1 ≤ h → ∀ r : ZMod (3 ^ h), r.val % 3 ≠ 0 →
761     c ≤ ndSyracuseUnitReferenceDensity h r := by
762     intro h hh r hr
763     cases h with
764     | zero => omega
765     | succ h =>
766       have hqh : q ≤ h := by omega
767       obtain ⟨v, z, hv, hz, he⟩ := bounded_inverse_into_cylinder q h hqh y r hr
768       have hzfloor := hfloor h hqh z hz
769       have hp : (1 / 2 : ℝ) ^ V ≤ (1 / 2 : ℝ) ^ (v : ℕ) :=
770         pow_le_pow_of_le_one (by norm_num) (by norm_num) hv

```

```

771   have hstep := referenceDensity_single_reverse_le h r z v he
772   apply le_trans _ hstep
773   dsimp [c]
774   calc
775     3 * (1 / 2 : ℝ) ^ v * a ≤ 3 * (1 / 2 : ℝ) ^ (v : ℕ) * a := by gcongr
776     _ ≤ 3 * (1 / 2 : ℝ) ^ (v : ℕ) * ndSyracuseUnitReferenceDensity h z := by gcongr
777   refine ⟨c, hc, ?_⟩
778   intro h hh r hr
779   by_cases hbig : q + 1 ≤ h
780   · exact hlarge h hbig r hr
781   · have hle : h ≤ q + 1 := by omega
782     classical
783     have hsum : (∑ z : ZMod (3 ^ (q + 1)),
784       if Tao.taoZModThreeProjection hle z = r then c else 0) ≤
785       ∑ z : ZMod (3 ^ (q + 1)), if Tao.taoZModThreeProjection hle z = r then
786       ndSyracuseUnitReferenceDensity (q + 1) z else 0 := by
787       apply Finset.sum_le_sum
788       intro z _
789       split_ifs with hz
790       · exact hlarge (q + 1) le_rfl z (unit_residue_of_projection hh hle z r hz hr)
791       · rfl
792     rw [projection_fiber_const, projection_fiber_density] at hsum
793     exact le_of_mul_le_mul_left hsum (by positivity : (0 : ℝ) < 3 ^ (q + 1 - h))
794
795   /-- Literal probability statement: the constant is independent of both
796   conductor and unit residue. The law is the imported Geom(2) Syracuse PMF. -/
797   theorem syracuse_uniform_atom_lower_bound :
798     ∃ c : ℝ, 0 < c ∧ ∀ n : ℕ, 1 ≤ n → ∀ r : ZMod (3 ^ n),
799     IsUnit r → c / (3 : ℝ) ^ n ≤ (Tao.syracPMF n r).toReal := by
800     obtain ⟨c, hc, hfloor⟩ := exists_uniform_positive_syracuse_density
801     refine ⟨(3 / 2) * c, by positivity, ?_⟩
802     intro n hn r hr
803     have hl : IsUnit (r.val : ZMod (3 ^ 1)) := by
804       have hh := hr.map (Tao.taoZModThreeProjection hn)
805       rwa [Tao.taoZModThreeProjection_eq_natCast_val] at hh
806     have hres : r.val % 3 ≠ 0 := by
807       intro hz
808       have hzero : (r.val : ZMod (3 ^ 1)) = 0 := by
809         apply ZMod.val_injective
810         simpa only [ZMod.val_natCast, pow_one, ZMod.val_zero] using hz
811       rw [hzero] at hl
812       change IsUnit (0 : ZMod 3) at hl
813       exact not_isUnit_zero hl
814     have hh := hfloor n hn r hres
815     unfold ndSyracuseUnitReferenceDensity ndSyracuseUniformDensity Tao.syracPMFMassVector at hh
816     push_cast at hh
817     apply (div_le_iff (by positivity : (0 : ℝ) < 3 ^ n)).2
818     nlinarith
819
820   end
821   end CollatzTargetDensity.SyracuseMinorant
822   -- END SyracuseCylinderSpread.lean
823
824   #print axioms CollatzTargetDensity.SyracuseMinorant.syracuse_uniform_atom_lower_bound

```